

MANUAL DE USO DE FLUXO: GUÍA PARA OBTENER RESPUESTAS COMPLETAS Y FIABLES

Este manual recoge las mejores prácticas para interactuar con Fluxo y maximizar la calidad, precisión y utilidad de las respuestas generadas por el asistente virtual, tanto en la consulta de documentación (*chatbot*) como en la resolución guiada de ejercicios (*copilot*). Se recogen directrices de *prompting* óptimo, formato de entrada, estructura de problemas y verificación, adaptadas a las características técnicas y pedagógicas de Fluxo.



ESCUELA POLITÉCNICA SUPERIOR GRADO EN INGENIERÍA MATEMÁTICA

Autor: Ignacio de Lecea Jiménez

Contenido

1.	Prompting óptimo en Fluxo	3
1.1.	Estructura de un prompt	3
1.2.	Formato Markdown y presentación de tablas.....	3
1.3.	Estructuración de la solución: pasos y verificación.....	4
1.4.	Redacción de enunciado: estructura y especificidad	6
2.	Instrucciones sintácticas y terminológicas	6
2.1.	Resumen de buenas prácticas	7
3.	Ejemplo de prompt óptimo para Fluxo.....	8

1. Prompting óptimo en Fluxo

Principios fundamentales del prompting:

- El *prompt* es la especificación detallada que se proporciona al modelo. Cuanto más preciso y estructurado sea, mayor será la calidad y fiabilidad de la respuesta.
- Un *prompt* óptimo debe contener cinco elementos: rol, instrucciones, contexto, input y output esperado.

El rol, las instrucciones y el output, no dependen del usuario, el input claramente sí, es lo que se introduce en los espacios de ENUNCIADO Y SOLUCIÓN.

Sin embargo, el contexto también depende del usuario, ya que en el input (ENUNCIADO) se puede especificar la contextualización SIEMPRE.

Por otro lado, está el output esperado, el cual es lo que esperamos que nos conteste Fluxo. Si esto no se cumple al completo, realizar un nuevo prompt especificándoselo en el ENUNCIADO.

1.1. Estructura de un prompt

1. **Rol:** “Eres un experto en gramáticas formales y autómatas finitos.”
2. **Instrucciones:** TEMPLATE_ANSWER Y TEMPLATE_PROBLEM.
3. **Input:** ENUNCIADO + SOLUCIÓN
4. **Contexto:** “Utiliza el método de subconjuntos para la conversión de AFND a AFD.”
5. **Output esperado:** “Incluye la verificación de la eliminación de la recursividad indirecta.”

1.2. Formato Markdown y presentación de tablas

- Siempre emplear formato **Markdown** para estructurar la información, encabezados y tablas.
- Los autómatas y máquinas de Turing deben representarse siempre en el siguiente formato de tabla:

El AFD se representa en la siguiente tabla:

Estado	Entrada 'a'	Entrada 'b'
q0	q0, q1	q0
q1	q2	-
q2	-	q2

- Utilizar siempre símbolos matemáticos en **texto plano**, según el glosario de Fluxo.

Expresión	Texto Plano	Ejemplo de uso
cdot	·	$A \cdot B$
dot{x}	$\dot{x}$	$\dot{x}$
dots	...	$a_1, a_2, \dots, a_n$
cup	$\cup$	$A \cup B$
cap	$\cap$	$A \cap B$

1.3. Estructuración de la solución: pasos y verificación

- Desglosar la solución en pasos **claramente identificables** con encabezados Markdown:
 - # Paso 1
 - ## 1.1
 - # Paso 2
 - etc.
- Seguir los pasos de los **algoritmos** desarrollados en la base de conocimiento de Fluxo (se pueden consultar desde la funcionalidad de documentación).

Algoritmos ya implementados:

- Conversión de un Autómata Finito No Determinista (AFND) a un Autómata Finito Determinista (AFD) (Método de Subconjuntos).
- Conversión de un Autómata de Pila No Determinista (APND) en una Gramática Independiente del Contexto (GIC).
- Algoritmo Cocke-Younger-Kasami (CYK).
- Conversión de una Expresión Regular (ER) a Gramática Regular (GR) (Método de las derivadas).

- Conversión de una Expresión Regular (ER) a Gramática Regular (GR) a Autómata Finito (AF) (Método de las derivadas).
- Factorización a Izquierdas para Gramática Independiente del Contexto (GIC).
- Conversión de una Gramática Independiente de Contexto (GIC) a Forma Normal de Chomsky (FNC).
- Conversión de una Gramática Independiente de Contexto (GIC) a Forma Normal de Greibach (FNG).
- Conversión de una Gramática Independiente de Contexto (GIC) a un Autómata de Pila No Determinista (APND).
- Conversión de una Gramática Lineal por la Derecha (GLD) a Gramática Lineal por la Izquierda (GLI).
- Conversión de una Gramática Lineal por la Izquierda (GLI) a Gramática Lineal por la Derecha (GLD).
- Simplificación de una Gramática Independiente de Contexto (GIC) (eliminación de no generativos, inaccesibles, λ -producciones y unitarias).
- Minimización de un Autómata Finito Determinista (AFD).
- Eliminación de la Recursividad por la Izquierda en un paso.
- Eliminación de la Recursividad por la Izquierda en múltiples pasos (directa e indirecta).
- Algoritmo de Thompson: Conversión de Expresiones Regulares (ER) en Autómatas Finitos No Deterministas (AFND))
- Incluir siempre un paso de **verificación** al final de la solución, comprobando explícitamente que:
 - Se han eliminado recursividades (directa e indirecta) si procede.
 - La gramática cumple las propiedades requeridas (GR, GIC, FNC, etc.).
 - No quedan producciones inútiles o inaccesibles.
 - El autómata o la gramática final cumple las condiciones del lenguaje.

1.4. Redacción de enunciado: estructura y especificidad

- **Estructurar siempre el enunciado en dos niveles:**

- # Problema: Descripción genérica del tipo de tarea.
- ## Enunciado: Descripción específica con todos los datos y condiciones.

Ejemplo:

Problema: *Análisis y simplificación de una gramática.*

Enunciado:

Dada la gramática GG:

- $S \rightarrow AB/AC/BCS \rightarrow AB/AC/BC$
- $A \rightarrow BC/aA \rightarrow BC/a$
- $B \rightarrow AC/bB \rightarrow AC/b$
- $C \rightarrow AB/cAC \rightarrow AB/cA$

Determine las propiedades del lenguaje generado por la gramática y realice un análisis de su estructura.

- Es una buena práctica, aunque no necesario, añadir siempre en el enunciado todo lo que se desea que Fluxo **verifique explícitamente**, por ejemplo:
 - Que la gramática esté en FNC antes de aplicar CYK.
 - Que la simplificación ha eliminado todas las producciones inútiles.
 - Que la conversión cumple las propiedades requeridas.
 - Que se ha eliminado la recursividad directa e indirecta.

2. Instrucciones sintácticas y terminológicas

- Utilizar vocabulario técnico de manera rigurosa y coherente: Gramática Independiente del contexto (*GIC*), Gramática regular (*GR*), Autómata finito no determinista (*AFND*), etc.
- Ser preciso y específico en la descripción de pasos, condiciones y resultados.
- No introducir **ambigüedades**, por ejemplo:

Determinar si una cadena pertenece a un lenguaje usando CYK.

Siempre hay que comprobar si la gramática inicial está en FNC, pues no decir en ningún punto, ni en enunciado ni en solución, que la gramática ya se ha

comprobado que está en FNC, ya que una instrucción de Fluxo es comprobar antes de aplicar el algoritmo CYK si la gramática inicial está en FNC, porque esto es una instrucción **ambigua**, le decimos que lo compruebe y a la vez que ya se ha comprobado, por tanto muchas veces no lo comprobará.

- **Verificación y control de calidad**

- No omitir nunca el paso de verificación: si el ejercicio requiere comprobar propiedades, estas deben incluirse explícitamente en el enunciado y desarrollarse en la solución.
- Evitar suposiciones no justificadas: nunca asumir que una gramática está en FNC, o que una propiedad se cumple, si no se ha verificado explícitamente.

2.1. Resumen de buenas prácticas

- Formular prompts completos, estructurados y específicos.
- Utilizar formato Markdown para toda la entrada y salida.
- Presentar tablas y símbolos matemáticos según los estándares definidos.
- Seguir la estructura de pasos de los algoritmos implementados.
- Incluir verificación exhaustiva en cada problema.
- Ser riguroso en la terminología y evitar ambigüedades.
- Añadir en el enunciado todas las verificaciones que se desean obtener.

3. Ejemplo de prompt óptimo para Fluxo

Asistente Virtual
Ingrese el enunciado del problema:

Ingrese su solución completa:

Evaluar Solución

Problema: Eliminación de la recursividad por la izquierda en una gramática independiente de contexto (GIC)

Enunciado

Elimina la recursividad por la izquierda (directa e indirecta) de la siguiente gramática independiente de contexto (GIC):

- $S \rightarrow SaB \mid SBa \mid ab$

- $A \rightarrow Baa \mid Cbc$

- $B \rightarrow CA b \mid CbA \mid ac$

- $C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Solucion

Paso 1: Gramática original

$S \rightarrow SaB \mid SBa \mid ab$

$A \rightarrow Baa \mid Cbc$

$B \rightarrow CA b \mid CbA \mid ac$

$C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Paso 2: Detección de problemas

- Recursividad directa:

Regla de S: $S \rightarrow SaB \mid SBa$

- Recursividad indirecta:

Ciclo entre no terminales ($A \rightarrow B \rightarrow C \rightarrow S \rightarrow A$)

Paso 3: Procedimiento de eliminación

3.1 Ordenar no terminales

Jerarquía según dependencias:

`S (nivel 1)  $\rightarrow$  A (nivel 2)  $\rightarrow$  B (nivel 3)  $\rightarrow$  C (nivel 4)`

3.2 Sustitución en cascada

1. Sustituir C en B:

$B \rightarrow AbbAb \mid SaaBAb \mid SbAb \mid ccAb$

$\mid AbbbA \mid SaaBbbA \mid SbbA \mid ccbA$

$\mid ac$

2. Sustituir B en A:

$A \rightarrow (CAb \mid CbA \mid ac)aa \mid Cbc$

3. Sustituir A en S:

(No requiere cambios adicionales)

3.3 Eliminar recursividad directa en S

Transformación estándar para producciones recursivas:

$S \rightarrow abS'$

$S' \rightarrow aBS' \mid BaS' \mid \varepsilon$

Paso 4: Gramática transformada final

$S \rightarrow abS'$

$S' \rightarrow aBS' \mid BaS' \mid \varepsilon$

$A \rightarrow Baa \mid Cbc$

$B \rightarrow CAb \mid CbA \mid ac$

$C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Paso 5: Verificación de resultados

- Recursividad directa eliminada:

Ninguna producción de S inicia con S

- Jerarquía preservada:

No hay ciclos entre no terminales

- Equivalencia garantizada:

Mismo lenguaje generado con estructura no recursiva

Se logra:

1. Eliminar la autorreferencia directa en `S`
2. Romper el ciclo indirecto $C \rightarrow SaaB \rightarrow B \rightarrow CAb \rightarrow C$

¿Por qué funciona sin reordenar no terminales?

1. La recursividad indirecta dependía de la producción $C \rightarrow SaaB$
2. Al modificar `S` para que nunca empiece con `S`, se evita que `C` pueda regenerar la recursividad

Conclusión

- En este caso particular, eliminar la recursividad directa en `S` fue suficiente
- En gramáticas más complejas con múltiples ciclos interdependientes, sería necesario el procedimiento completo